Inteligentný systém na automatizované testovanie bezpečnosti webových aplikácií pomocou veľkých jazykových modelov (AI-PENTEST): prehľad stavu v oblasti*

Kyrylo Bukhanets, Artur Toba, Maksym Tomkevych, Andrii Tsybulskyi
Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií
bukhanets@stuba.sk, xtoba@stuba.sk, xtomkevych@stuba.sk, xtribulskyi@stuba.sk

11. október 2026

Abstrakt

Veľké jazykové modely dokážu plánovať a vykonávať úlohy spojené s analýzou webových aplikácií, ich výsledky však treba hodnotiť bezpečne, reprodukovateľne a podľa jednotných kritérií. Článok poskytne prehľad existujúcich prístupov k automatizovanému bezpečnostnému testovaniu a k hodnoteniu agentných systémov. Osobitná pozornosť bude venovaná návrhu prostredia Web Pentest Harness, v ktorom AI agent pracuje s izolovanými zraniteľnými aplikáciami, všetky jeho akcie sa zaznamenávajú a nálezy sa overujú voči referenčnej databáze zraniteľností. Porovnanie bude založené na metrikách presnosti, úplnosti, miery falošne pozitívnych nálezov a času potrebného na nájdenie a overenie zraniteľnosti.

1 Úvod

Článok sa zameria na možnosti a obmedzenia veľkých jazykových modelov pri automatizovanom testovaní bezpečnosti webových aplikácií. Primárnym zdrojom, ktorý nás k téme usmernil, je OWASP Web Security Testing Guide [8]: definuje, čo všetko má úplný test webovej aplikácie pokryť, a tým aj to, voči čomu treba merať AI agentov. Ďalším východiskom sú požiadavky na systematické hodnotenie systémov umelej inteligencie [4, 5].

Webové aplikácie patria medzi najčastejšie napádané časti informačných systémov. Manuálne penetračné testovanie je drahé, závisí od skúseností testera a ťažko sa opakuje pri každej zmene aplikácie. Klasické automatizované skenery sú rýchle, no mnohé zraniteľnosti, ktoré vyžadujú porozumenie logike aplikácie, neodhalia. Veľké jazykové modely (angl. *large language models*, LLM) sľubujú prepojenie týchto dvoch svetov: dokážu interpretovať odpovede aplikácie, plánovať ďalšie kroky a ovládať bežné bezpečnostné nástroje.

*Predbežná verzia prehľadového článku v predmete Metódy inžinierskej práce, ak. rok 2026/27.

Rýchly vývoj agentných systémov však prináša aj otázku, ako ich schopnosti merať. Výsledky publikovaných prác sa ťažko porovnávajú, pretože každá používa iné testovacie aplikácie, iné nástroje a iné kritériá úspechu. Cieľom článku je preto (1) zhrnúť existujúce prístupy k automatizovanému bezpečnostnému testovaniu, (2) navrhnúť prostredie Web Pentest Harness, ktoré umožní bezpečné a opakovateľné experimenty, a (3) definovať metriky, podľa ktorých možno agentov objektívne porovnať.

Článok je členený nasledovne. Kapitola 2 opisuje súčasný stav v oblasti, kapitola 3 predstavuje návrh prostredia a kapitola 4 bezpečný priebeh experimentu. Metodiku hodnotenia uvádza kapitola 5, obmedzenia rozoberá kapitola 6 a kapitola 7 článok uzatvára.

2 Súčasný stav v oblasti

Táto časť porovná tradičné automatizované bezpečnostné nástroje, prístupy založené na veľkých jazykových modeloch a agentné riešenia. Kategórie testovaných zraniteľností budú vychádzať najmä z OWASP Top 10 [6].

2.1 Tradičné automatizované nástroje

Statická analýza (SAST) skúma zdrojový kód bez jeho spustenia a dokáže včas upozorniť na nebezpečné konštrukcie, často však hlási veľa falošne pozitívnych nálezov a nemá k dispozícii kontext nasadenia. Dynamická analýza (DAST) testuje bežiacu aplikáciu ako čiernu skrinku: prechádza jej stránky, posiela upravené požiadavky a vyhodnocuje odpovede. Typickým voľne dostupným nástrojom je ZAP [9]. Už staršia porovnávacia štúdia ukázala, že čiernoskrinkové skenery spoľahlivo nachádzajú jednoduché reflektované chyby, no zlyhávajú pri zraniteľnostiach viazaných na stav aplikácie, viacstupňové formuláre alebo logiku autorizácie [2]. Práve tieto prípady dnes rieši najmä človek – penetračný tester.

2.2 Prístupy založené na veľkých jazykových modeloch

Prvé práce využívali LLM ako asistenta, ktorý testerovi radí ďalší krok. Nástroj PentestGPT rozdeľuje prácu do samostatných modulov na uvažovanie, generovanie príkazov a spracovanie výstupov, čím zmierňuje stratu kontextu pri dlhých testoch [1]. Autori zároveň ukázali, že modely dobre zvládajú jednotlivé podúlohy, no bez štruktúry sa strácajú v celkovom pláne testu.

Ďalším krokom sú autonómni agenti, ktorí sami volajú nástroje, napríklad prehliadač alebo terminál, a podľa výsledkov upravujú svoj postup. Fang a kol. demonštrovali, že agent postavený na silnom modeli dokáže v izolovanom prostredí bez predchádzajúcej znalosti zraniteľnosti zneužiť viaceré typy chýb webových aplikácií vrátane SQL injection, zatiaľ čo slabšie modely v rovnakej úlohe zlyhávali [3].

Na porovnávanie agentov vznikajú benchmarky. Cybench zhromažďuje profesionálne úlohy typu *capture the flag* a delí ich na menšie podúlohy, aby bolo možné merať aj čiastočný pokrok agenta [10]. Takéto benchmarky sa však zameriavajú na nájdenie vlajky, nie na úplnosť a presnosť nálezov v reálnej webovej aplikácii, čo je medzera, ktorú má pokryť navrhovaný Web Pentest Harness.

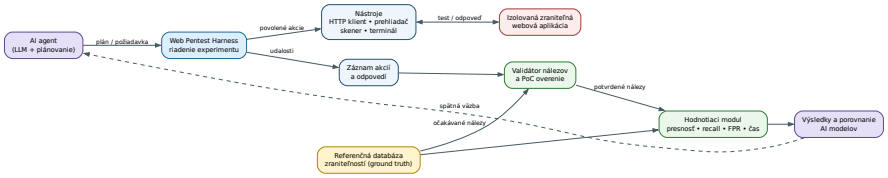
Hlavné rozdiely medzi prístupmi zhŕňa tab. 1.

Tabuľka 1: Porovnanie prístupov k automatizovanému testovaniu bezpečnosti.

Vlastnosť	SAST/DAST	LLM asistent	LLM agent
Rýchlosť	vysoká	stredná	stredná
Logika aplikácie	nízka	stredná	vyššia
Samostatnosť	úplná	nízka	vysoká
Falošné nálezy	časté	závisí od testera	časté
Opakovateľnosť	vysoká	nízka	nízka

3 Návrh Web Pentest Harness

Navrhované riešenie oddelí rozhodovanie AI agenta od nástrojov, testovanej aplikácie a hodnotiaceho modulu. Architektúru a tok údajov znázorňuje obr. 1.



Obr. 1: Navrhovaná architektúra prostredia Web Pentest Harness.

4 Bezpečný priebeh experimentu

Experimenty budú prebiehať výlučne v izolovanom prostredí so zámerne zraniteľnými aplikáciami, napríklad OWASP WebGoat [7]. Každá akcia agenta sa pred vykonaním skontroluje, zaznamená a prípadný nález sa overí kontrolovaným proof-of-concept postupom podľa obr. 2.

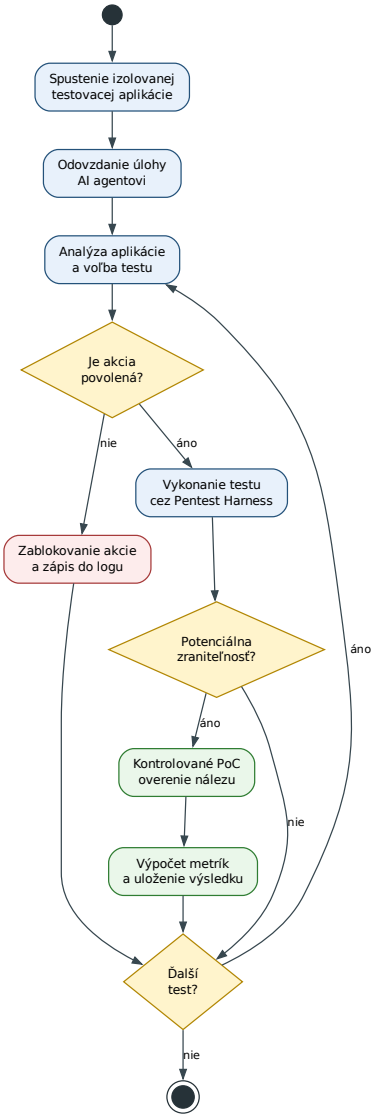
5 Metodika hodnotenia

Hodnotenie bude porovnávať nálezy agenta s referenčnou databázou známych zraniteľností. Sledované budú presnosť, úplnosť, miera falošne pozitívnych nálezov, čas do nájdenia chyby a úspešnosť jej kontrolovaného overenia.

Nález agenta sa považuje za správny (TP), ak sa zhoduje so záznamom v referenčnej databáze v type zraniteľnosti aj v jej umiestnení (URL a parameter). Nález, ktorý v databáze nie je, je falošne pozitívny (FP), a zraniteľnosť z databázy, ktorú agent nenašiel, je falošne negatívna (FN). Z týchto hodnôt vypočítame presnosť P, úplnosť R a ich harmonický priemer F_1:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}. \tag{1}$$

Mieru falošne pozitívnych nálezov vyjadríme ako podiel $FP/(TP+FP)$, teda $1-P$. Čas do nájdenia t_i meriame od spustenia behu po prvé správne nahlásenie i-tej zraniteľnosti; súhrnne uvedieme medián cez všetky nájdené zraniteľnosti. Keďže výstupy LLM nie sú deterministické, každý agent sa na každej aplikácii



Obr. 2: Diagram aktivít bezpečnostného testovania vytvorený v UMLet/UMLetino.

spustí viackrát a výsledky uvedieme ako priemer so smerodajnou odchýlkou. Okrem metrík sa zaznamená aj počet volaní nástrojov a spotreba tokenov, aby bolo možné porovnať aj nákladovosť agentov.

6 Diskusia a obmedzenia

Diskusia sa bude venovať reprodukovateľnosti experimentov, závislosti výsledkov od konkrétneho modelu a nástrojov, riziku falošných nálezov a bezpečnostným i etickým hraniciam testovania.

Reprodukovateľnosť. Komerčné modely sa priebežne aktualizujú, preto rovnaký experiment o niekoľko mesiacov nemusí dať rovnaký výsledok. Harness preto zaznamenáva presnú verziu modelu, parametre generovania, verzie nástrojov aj testovaných aplikácií a celý priebeh komunikácie agenta.

Závislosť od nástrojov. Výkon agenta neurčuje iba model, ale aj sada nástrojov, ktoré má k dispozícii, a spôsob, akým sú opísané. Pri porovnávaní modelov musí byť táto sada rovnaká, inak by sa merala kvalita integrácie, nie modelu.

Reprezentatívnosť. Zámerne zraniteľné aplikácie ako WebGoat [7] obsahujú známe a dobre zdokumentované chyby, ktoré sa mohli objaviť v trénovacích dátach modelov. Vysoké skóre preto nemusí znamenať úspech na neznámej aplikácii; tento efekt plánujeme zmierniť pridaním vlastných, nepublikovaných zraniteľností.

Bezpečnosť a etika. Autonómny agent schopný zneužiť zraniteľnosť je nástrojom dvojakého použitia. Experimenty preto prebiehajú iba v izolovanej sieti bez prístupu na internet, každá akcia prechádza kontrolou rozsahu a deštruktívne operácie sú zakázané. Výsledky budú publikované bez návodov na zneužitie konkrétnych reálnych systémov.

7 Záver

Záver zhrnie zistenia prehľadu a vyhodnotí, do akej miery môže štandardizovaný Web Pentest Harness podporiť objektívne porovnávanie AI agentov.

Literatúra

[1] Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. PentestGPT: Evaluating and harnessing large language models for automated penetration testing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 847–864, 2024.

[2] Adam Doupé, Marco Cova, and Giovanni Vigna. Why Johnny can't pentest: An analysis of black-box web vulnerability scanners. In *Detection of*

Intrusions and Malware, and Vulnerability Assessment (DIMVA 2010), volume 6201 of *Lecture Notes in Computer Science*, pages 111–131. Springer, 2010.

[3] Richard Fang, Rohan Bindu, Akul Gupta, Qiusi Zhan, and Daniel Kang. LLM agents can autonomously hack websites. arXiv:2402.06664, 2024.

[4] T. Jensen, M. Przybocki, M. Kodwani, K. Greene, P. Hall, R. Amironesei, and C. Greenberg. Aria evaluation planning manual: Elements of aria-style ai evaluations. Technical Report NIST AI 200-3, National Institute of Standards and Technology, 2026.

[5] National Institute of Standards and Technology. Ai measurement and evaluation. `https://www.nist.gov/ai-measurement-and-evaluation`, 2024. Online; citované 11. 10. 2026.

[6] OWASP Foundation. Owasp top 10: The ten most critical web application security risks. `https://owasp.org/www-project-top-ten/`. Online; citované 11. 10. 2026.

[7] OWASP Foundation. Owasp webgoat. `https://owasp.org/www-project-webgoat/`. Online; citované 11. 10. 2026.

[8] OWASP Foundation. Web security testing guide. `https://owasp.org/www-project-web-security-testing-guide/`, 2020. Online; citované 11. 10. 2026.

[9] ZAP Development Team. ZAP – zed attack proxy. `https://www.zaproxy.org/`. Online; citované 1. 11. 2026.

[10] Andy K. Zhang et al. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models. arXiv:2408.08926, 2024.